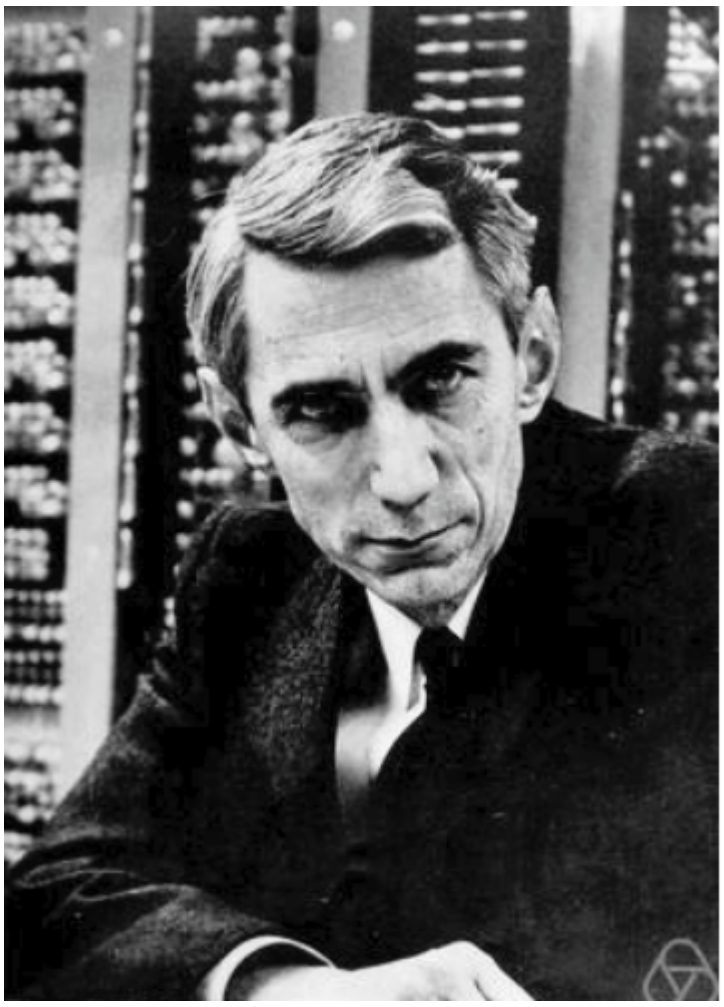
# Entropy Demystified
# 解密熵

Is it a disorder, uncertainty or surprise?
是无序、不确定还是惊喜?

The idea of entropy is confusing at first because so many words are used to describe it: disorder, uncertainty, surprise, unpredictability, amount of information and so on. If you've got confused with the word "entropy", you are in the right place. I am going to demystify it for you.
熵的概念一开始让人感到困惑，因为有太多的词汇被用来描述它：无序、不确定性、惊喜、不可预测性、信息量等等。如果你对 "熵 "这个词感到困惑，那你来对地方了。我将为你揭开它的神秘面纱。

## Who Invented Entropy and Why?
## 谁发明了熵，为什么?

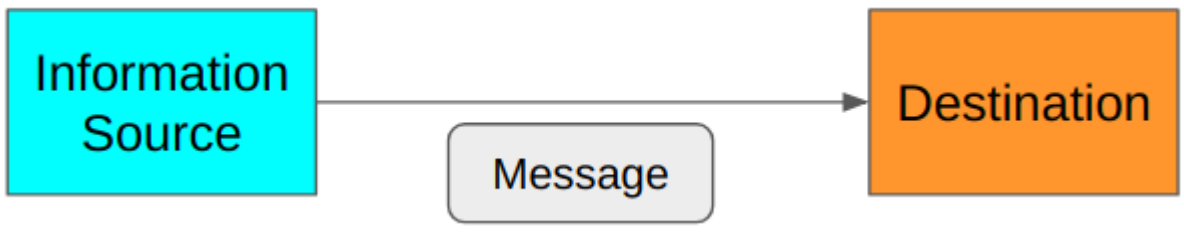
In 1948, Claude Shannon introduced the concept of information entropy in his paper "A Mathematical Theory of Communication".
1948 年，克劳德-香农在其论文《通信的数学理论》中提出了信息熵的概念。



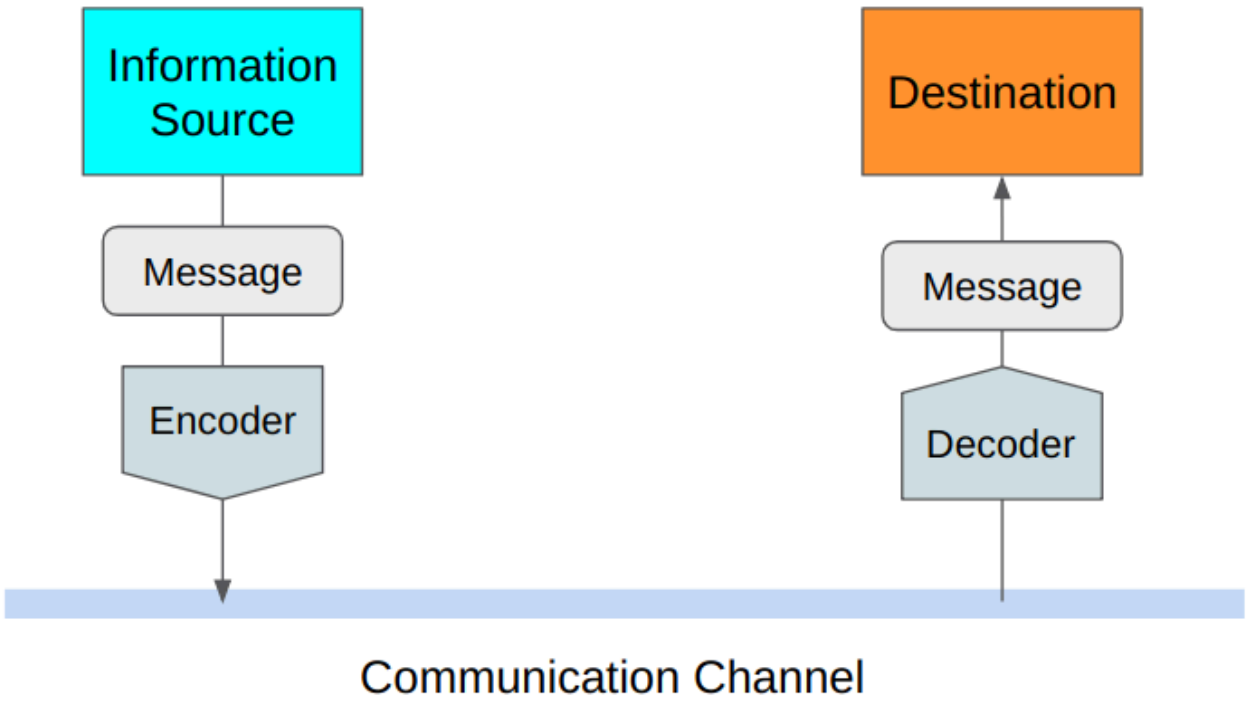
Claude Shannon
克劳德-香农

Source: https://en.wikipedia.org/wiki/Claude_Shannon
来源：https://en.wikipedia.org/wiki/Claude_Shannon

Shannon was looking for a way to efficiently send messages without losing any information.
香农一直在寻找一种既能高效发送信息，又不会丢失任何信息的方法。




Shannon measured the efficiency in terms of average message length. Therefore, he was thinking about how to encode the original message into the smallest possible data structure. At the same time, he required that there should be no information loss. As such, the decoder at the destination must be able to restore the original message losslessly.
香农用平均信息长度来衡量效率。因此，他考虑的是如何将原始信息编码成尽可能小的数据结构。同时，他还要求不能有信息丢失。因此，目的地的解码器必须能够无损地恢复原始信息。

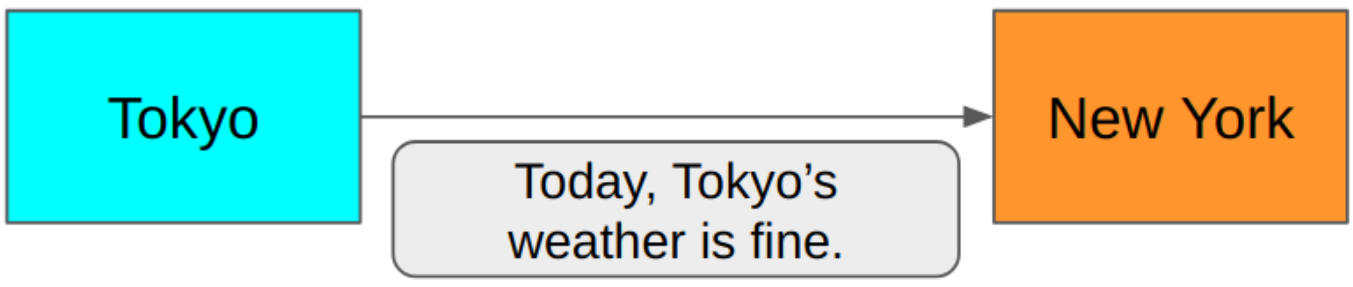



Shannon defined the entropy as the smallest possible average size of lossless encoding of the messages sent from the source to the destination. He showed how to calculate the entropy which is a useful thing to know as to make efficient use of the communication channel.
香农将熵定义为从信源发送到目的地的无损编码信息的最小可能平均大小。他展示了如何计算熵，这对有效利用通信信道非常有用。

The above definition of the entropy might not be obvious to you at this moment. We shall see a lot of examples to learn what it means to efficiently and losslessly send messages.
此时此刻，您可能还不太清楚上述熵的定义。我们将通过大量实例来了解高效、无损地发送信息意味着什么。

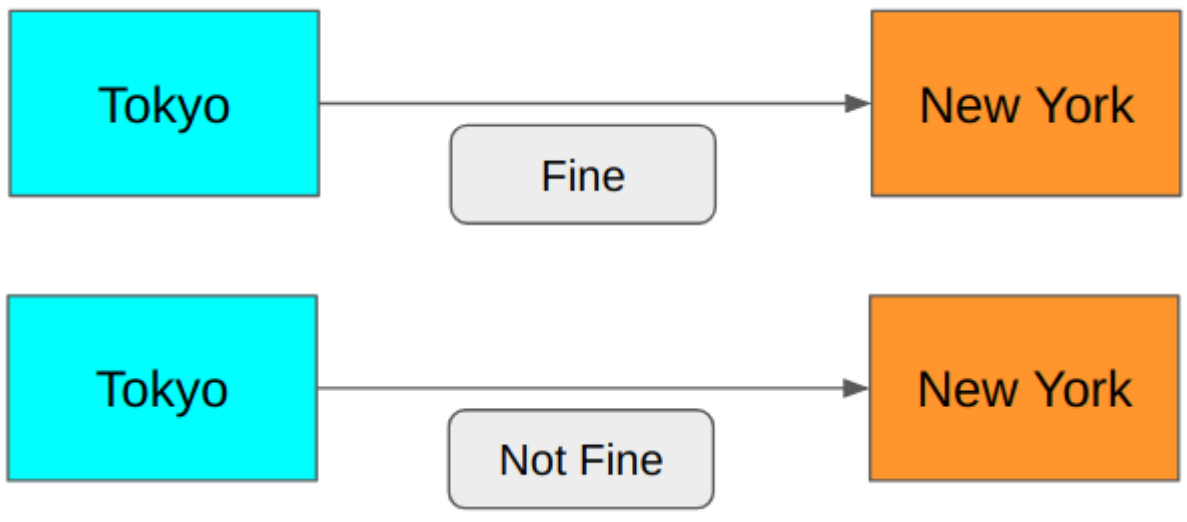
## How to Make Efficient and Lossless Encoding?
## 如何进行高效无损编码?

Suppose you want to send a message from Tokyo to New York regarding Tokyo's weather today.
假设您想从东京向纽约发送一条关于东京今天天气的信息。



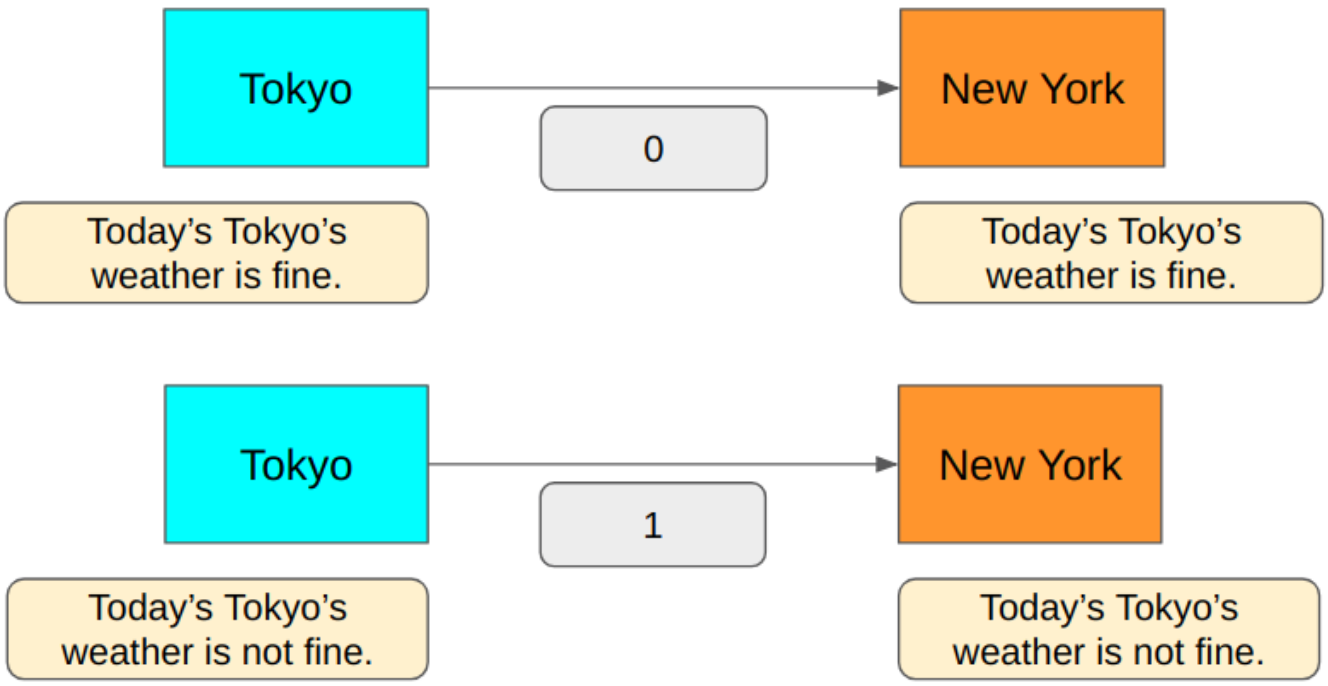

Is this efficient?
这样做有效吗?

In this scenario, the sender in Tokyo and the receiver in New York both know that they are only communicating today's Tokyo's weather. So, we don't need to send the words like "Today", "Tokyo's" and "weather". We need to tell "Fine" or "Not fine", and that's it.
在这种情况下，东京的发送方和纽约的接收方都知道，他们只是在交流今天东京的天气情况。因此，我们不需要发送 "今天"、"东京 "和 "天气 "等词语。我们只需告知 "很好 "或 "不好"，仅此而已。




It is much shorter. Can we do better?
它要短得多。我们能做得更好吗?

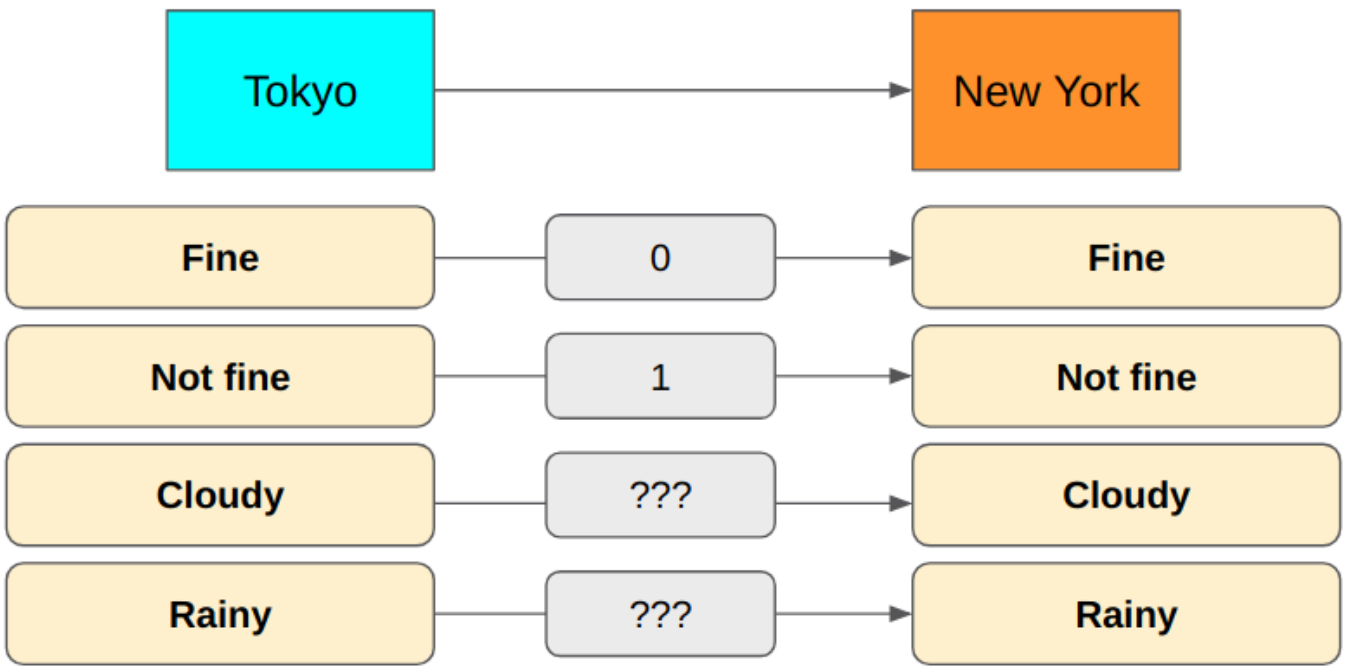
You bet. Because we only need to tell "yes" or "no" which is a binary. We need precisely 1 bit to encode the above messages. 0 for "Fine" and 1 for "Not fine".
没错。因为我们只需要知道 "是 "或 "否"，这是二进制。我们恰好需要 1 位来编码上述信息。0 表示 "好"，1 表示 "不好"。



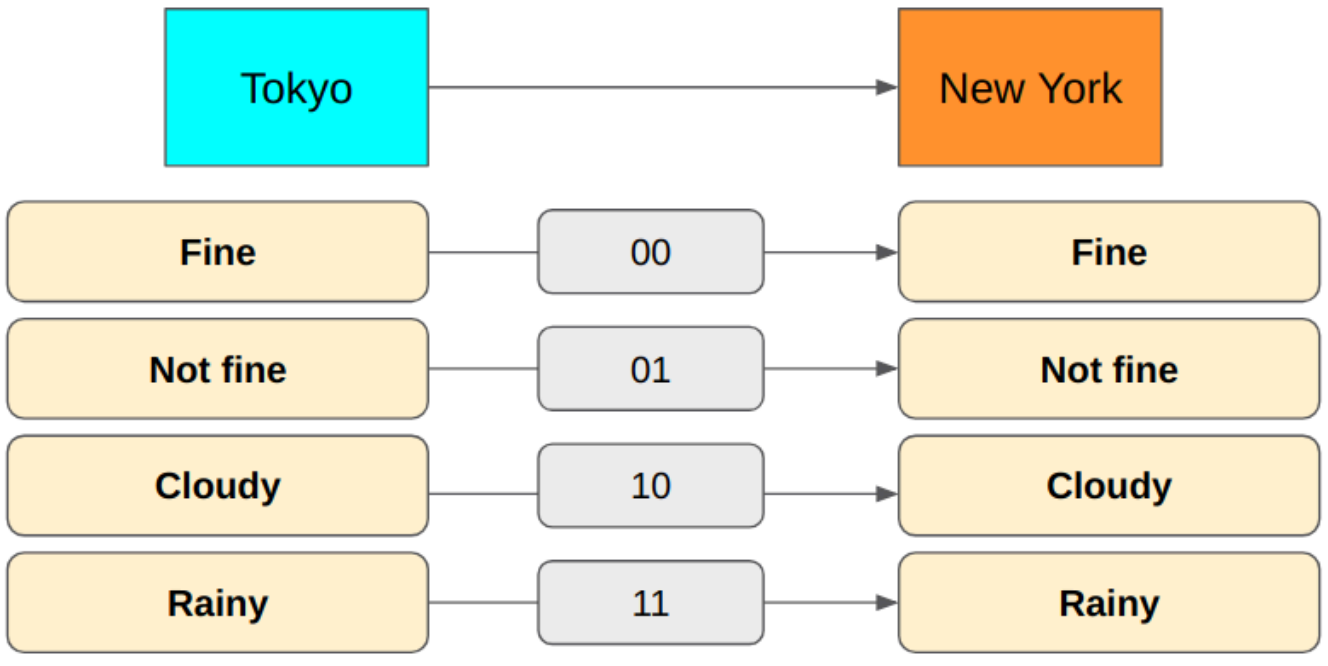

Is this lossless?
这是无损的吗?

Yes, we can encode and decode the message without losing any information since we only have two possible message types: “Fine” and “Not fine”.
是的，我们可以在不丢失任何信息的情况下对信息进行编码和解码，因为我们只有两种可能的信息类型：好 "和 "不好"。

But if we also want to talk about “Cloudy” or “Rainy”, the 1-bit encoding won’t be able to cover all cases.
但如果我们还想讨论 "多云 "或 "下雨"，1 位编码就无法涵盖所有情况。



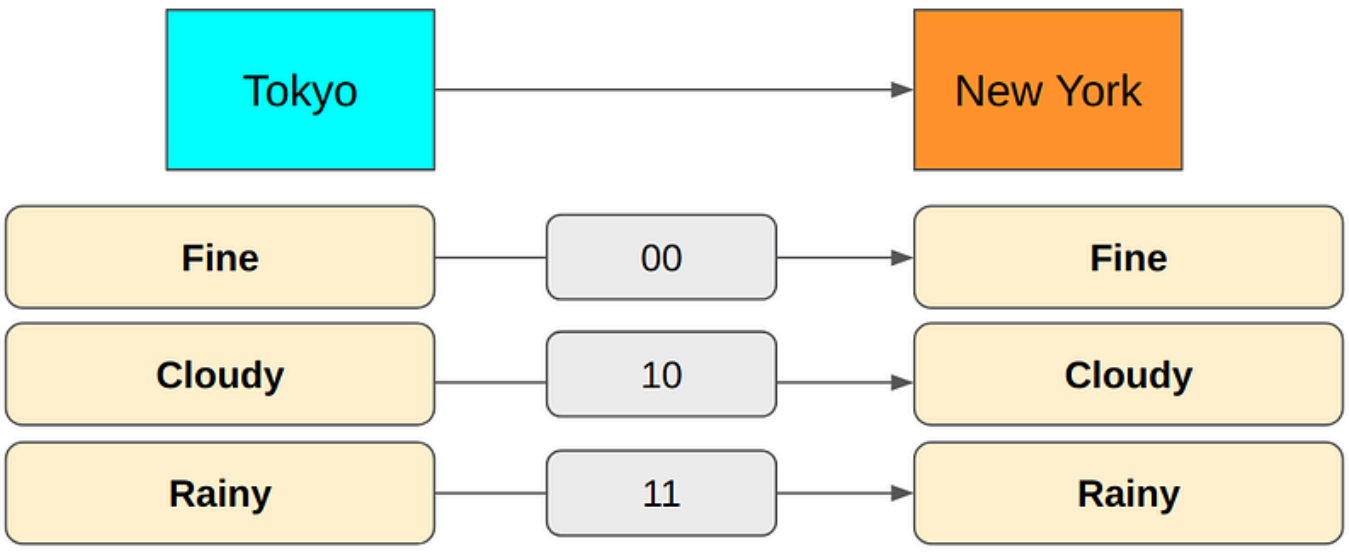

How about using 2 bits instead?
用 2 位来代替如何?



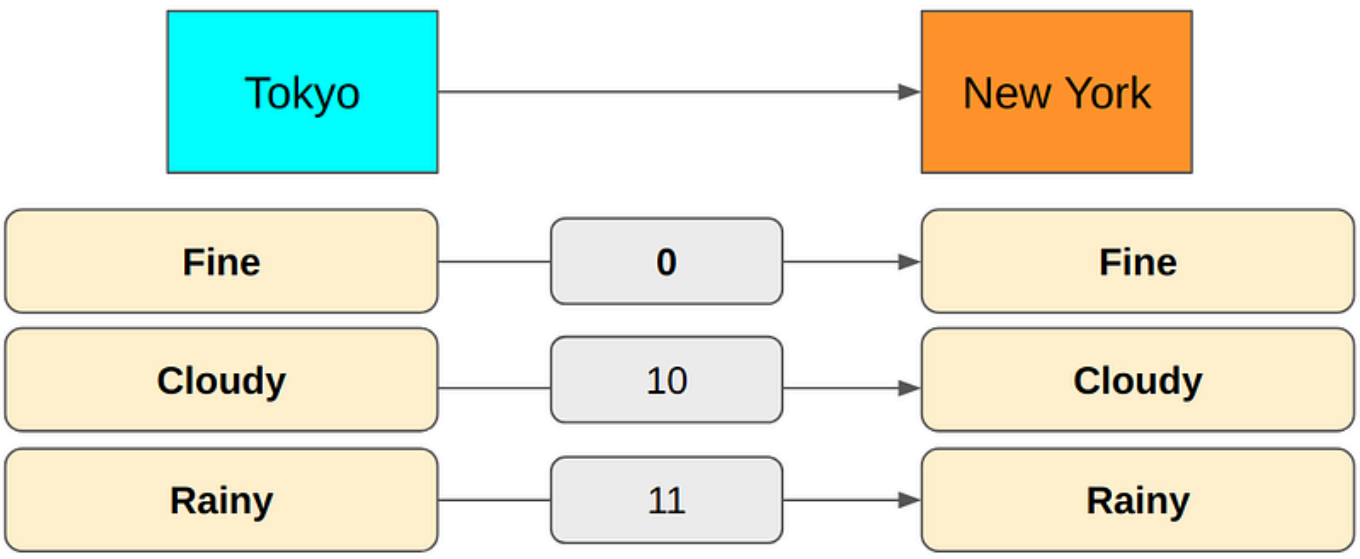

Now we can express all cases. It requires only 2 bits to send the messages.
现在，我们可以表达所有情况。发送信息只需要 2 个比特。

But there is something wrong with this approach. Semantically, “Cloudy” and “Rainy” are both “Not fine” as well. It seems “Not fine” is redundant. Let’s remove it.

但这种方法有问题。从语义上讲，"阴天 "和 "下雨 "也都是 "不好"。看来 "不好 "是多余的。把它去掉吧。



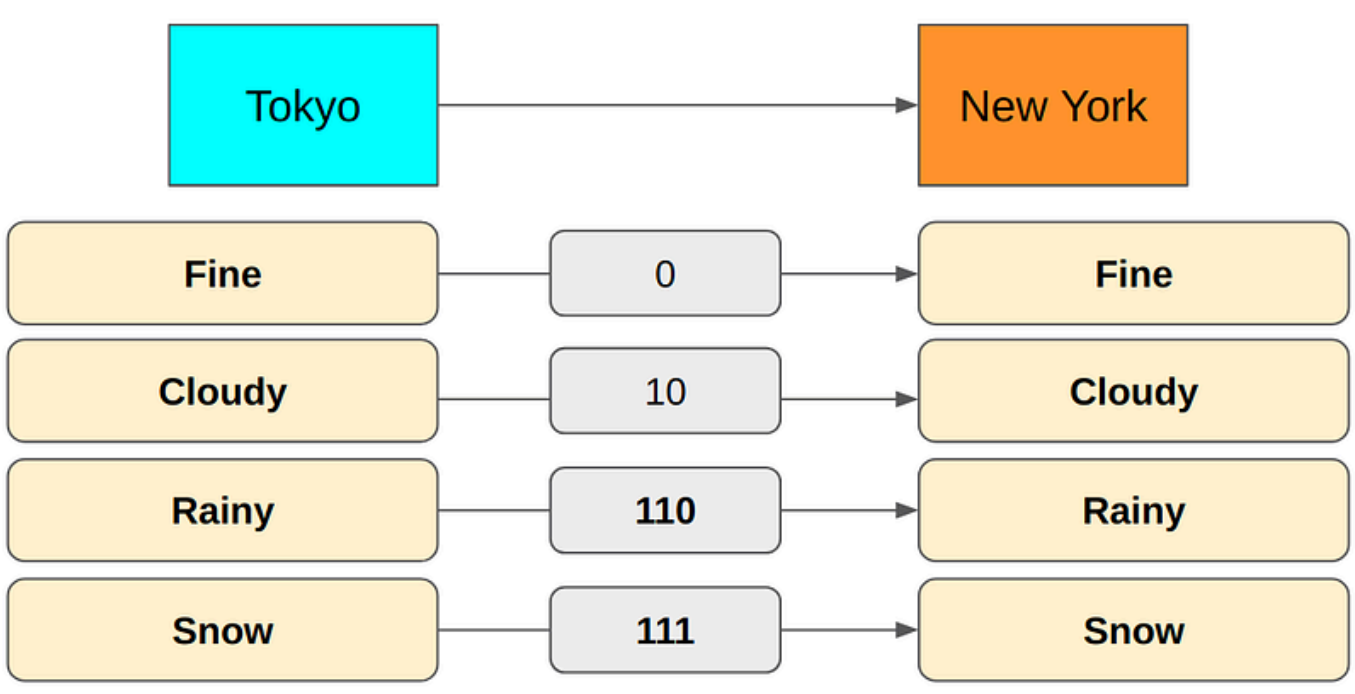

In the above encoding, “Fine” is “00” but we don’t need the second zero. When the first bit is zero, we already know it is “Fine” because both “Cloudy” and “Rainy” start with “1”.
在上述编码中，"Fine "是 "00"，但我们不需要第二个 0。当第一位为 0 时，我们已经知道它是 "Fine"，因为 "Cloudy "和 "Rainy "都以 "1 "开头。




In the above encoding, the first bit indicates “Fine or not”. The second bit indicates “Cloudy or Rainy”.
在上述编码中，第一位表示 "晴或不晴"。第二位表示 "多云或阴雨"。

But there is a problem. It can snow in Tokyo, and we have no way to communicate such weather. What do we do?
但有一个问题。东京可能会下雪，而我们却无法传达这样的天气。怎么办呢?

We could add a third bit and use it for “Rainy” and “Snow” cases.
我们可以添加第三个位，用于 "雨天 "和 "雪天 "的情况。




We changed “Rainy” from 11 to 110 and added “Snow” as 111 because 110 is distinctly different from 111, but 11 is ambiguous when multiple encodings are sent one after another.
我们将 "Rainy "从 11 改为 110，并将 "Snow "添加为 111，因为 110 与 111 有明显的不同，但当多种编码相继发送时，11 就会产生歧义。

For example, if we use 11 for “Rainy” and 111 for “Snow”, a 5-bit value “11111” could mean either “Rainy, Snow” or “Snow, Rainy”. Because of this, we use 110 for “Rainy” so that we can use “110111” for “Rainy, Snow” and “111110” for “Snow, Rainy”. There is no ambiguity.
例如，如果我们用 11 表示 "雨天"，用 111 表示 "雪天"，那么 5 位数值 "11111 "既可以表示 "雨天，雪天"，也可以表示 "雪天，雨天"。因此，我们用 110 表示 "Rainy"，这样就可以用 "110111 "表示 "Rainy, Snow"，用 "111110 "表示 "Snow, Rainy"。这样就不会产生歧义。

So, the encoding now uses the first bit for “Fine or not”, the second bit for “Cloudy or others” and the third bit for “Rain or snow”.
因此，现在的编码使用第一个比特表示 "晴或不晴"，第二个比特表示 "多云或其他"，第三个比特表示 "雨或雪"。

For example, “010110111” means “Fine, Cloudy, Rainy, Snow”. “110010111” means “Rainy, Fine, Cloudy, Snow”. Once again, there is no ambiguity. It is lossless, and it seems very efficient.
例如，"010110111 "表示 "晴、多云、雨、雪"。"110010111 "表示 "雨、晴、多云、雪"。再一次，没有任何歧义。它是无损的，而且似乎非常有效。

There are more weather conditions in Tokyo than just those four cases but let’s keep the problem simple for now.
东京的天气情况不止这四种，但我们还是先把问题简单化吧。

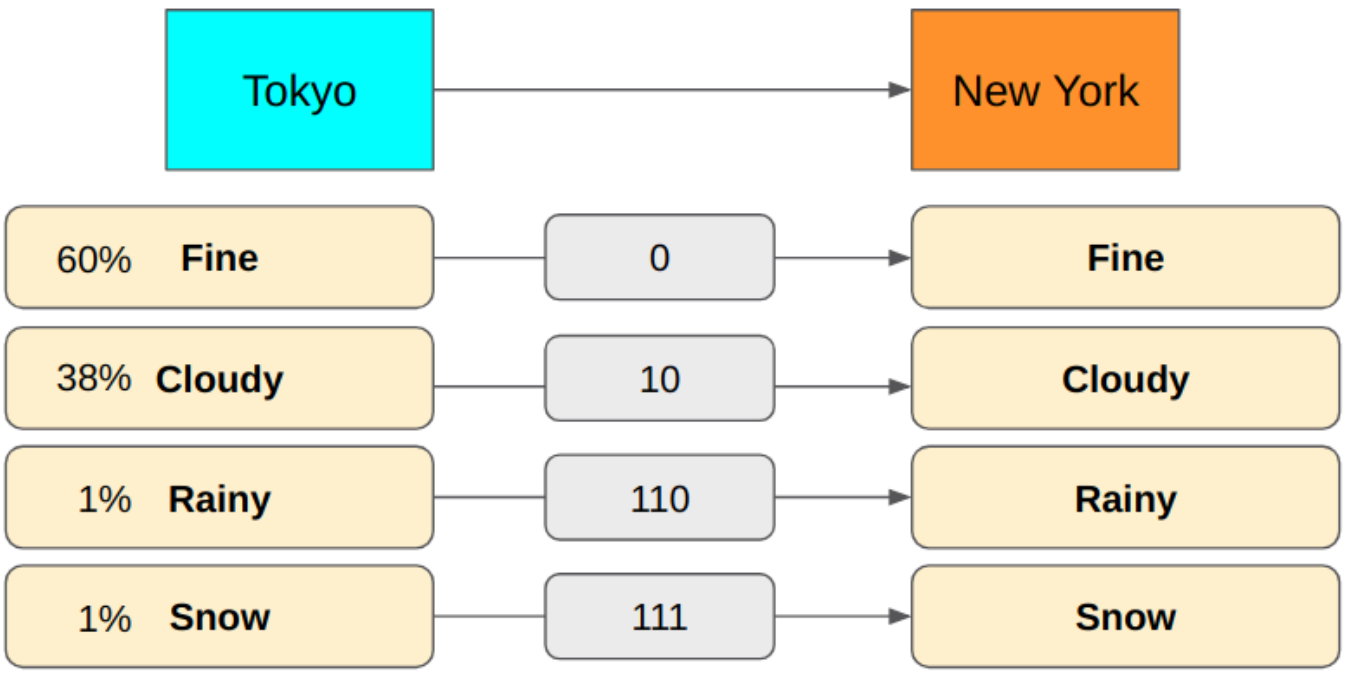
So, are we done?
那么，我们说完了吗?

Not quite — we want to know whether we’ve achieved the smallest possible average encoding size using the above lossless encoding or not. How do we calculate the average encoding size anyway?
不完全是这样，我们想知道的是，使用上述无损编码后，我们是否实现了尽可能小的平均编码大小。我们到底该如何计算平均编码大小呢?

## How to Calculate Average Encoding Size
## 如何计算平均编码大小

Let’s suppose Tokyo is sending the weather messages to New York many times (say, every hour) using the lossless encoding we’ve discussed so far.
假设东京使用我们之前讨论过的无损编码方式多次（比如每小时一次）向纽约发送天气信息。

We can accumulate weather report data so that we know the probability distribution of message types sent from Tokyo to New York.
我们可以积累天气报告数据，从而了解从东京发送到纽约的信息类型的概率分布。

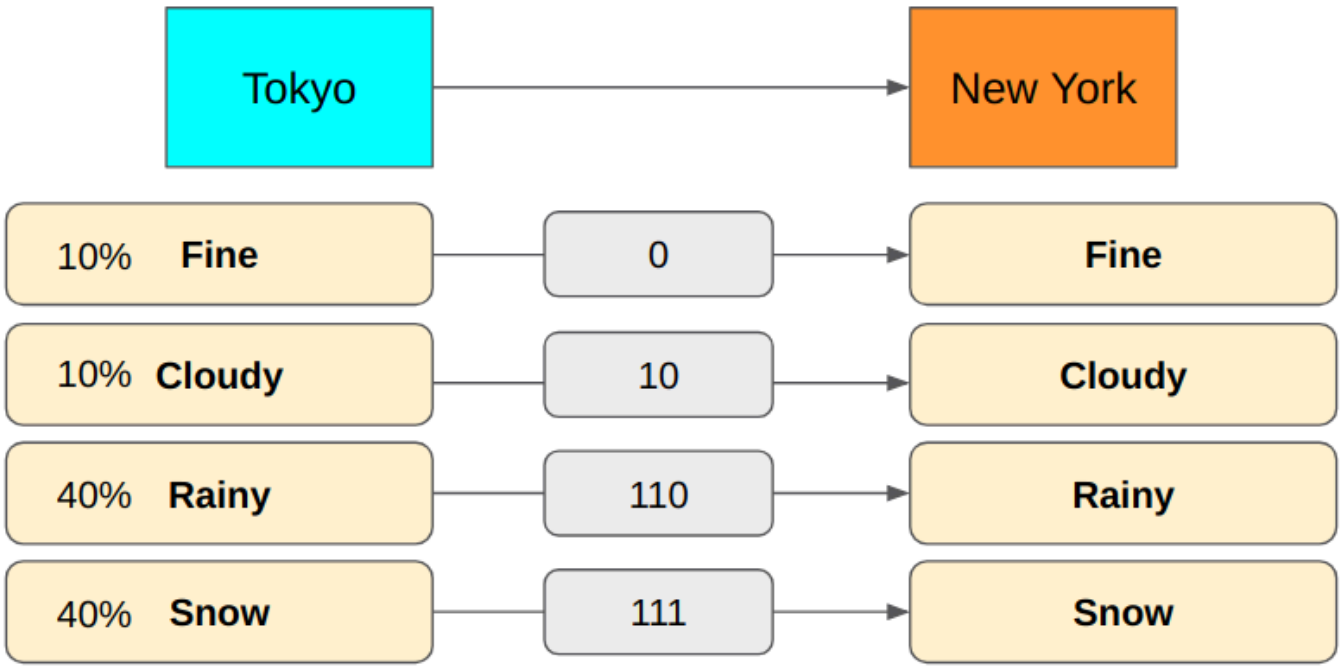



Then, we can calculate the average number of bits used to send messages from Tokyo to New York.
然后，我们就可以计算出从东京向纽约发送信息所使用的平均比特数。

(0.6 x 1 bit)+(0.38 x 2 bits)+(0.01 x 3 bits)+(0.01 x 3 bits)=1.42 bits
(0.6 x 1 位)+(0.38 x 2 位)+(0.01 x 3 位)+(0.01 x 3 位)=1.42 位

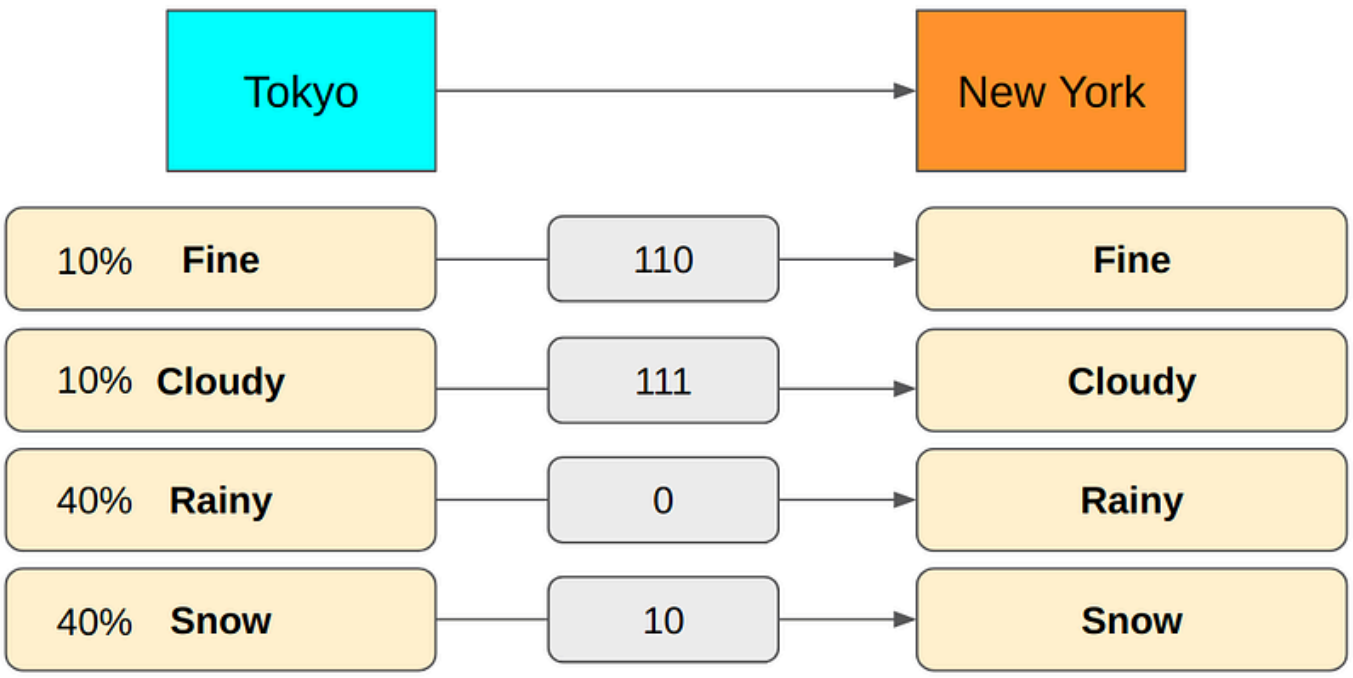
So, with the above encoding, we use 1.42 bits on average per message. As you can see, the average number of bits depends on the probability distribution and the message encoding (how many bits we use for each message type).
因此，在上述编码下，每条信息平均使用 1.42 比特。可以看出，平均比特数取决于概率分布和报文编码（每种报文类型使用多少比特）。

For example, if we have much more rain and snow in Tokyo, the average encoding size will be very different.
例如，如果东京的雨雪天气更多，平均编码大小就会大不相同。




(0.1 x 1 bit)+(0.1 x 2 bits)+(0.4 x 3 bits)+(0.4 x 3 bits)=2.7 bits
(0.1 x 1 位)+(0.1 x 2 位)+(0.4 x 3 位)+(0.4 x 3 位)=2.7 位

We would use 2.7 bits to encode messages on average. We could reduce the average encoding size by changing the encoding to use fewer bits for “Rainy” and “Snow”.
我们平均使用 2.7 比特对信息进行编码。我们可以通过改变编码来减少 "Rainy "和 "Snow "的平均编码量。



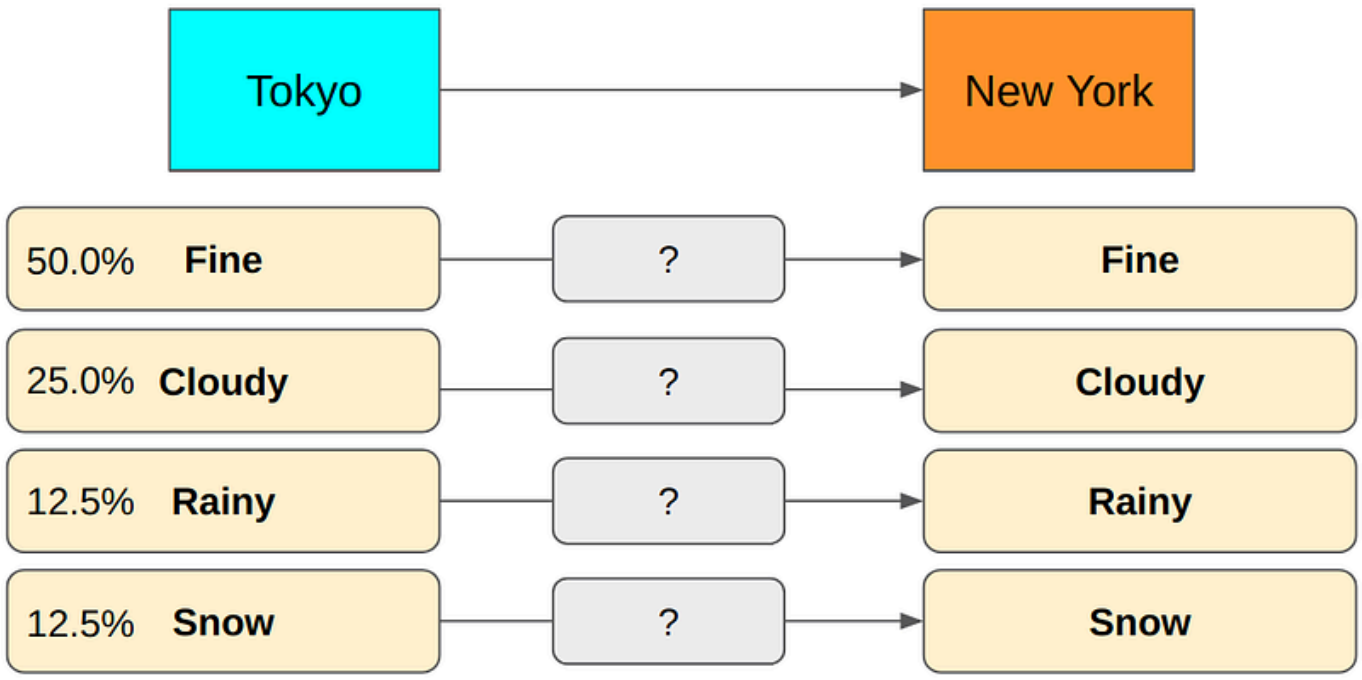

The above encoding requires 1.8 bits on average which is a much smaller size than what the previous encoding yields.
上述编码平均需要 1.8 比特，比前一种编码产生的大小要小得多。

Although this encoding yields smaller average encoding size, we lose the nice semantic meaning we had in the previous encoding scheme (i.e., the first bit for “Fine or not”, the second bit for, etc.). It’s ok since we are only concerned with the average encoding size, and we worry neither the semantics of encoding nor ease of decoding.
虽然这种编码方法产生的平均编码大小较小，但我们却失去了之前编码方案中的良好语义（即第一位表示 "好或不好"，第二位表示等）。没关系，因为我们只关心平均编码大小，既不担心编码的语义，也不担心解码的难易程度。

We now know how to calculate an average encoding size. But how do we know if our encoding is the best one that achieves the smallest average encoding size?
我们现在知道了如何计算平均编码大小。但我们如何知道我们的编码是否是实现最小平均编码大小的最佳编码呢?

## Finding the Smallest Average Encoding Size
## 寻找最小的平均编码大小

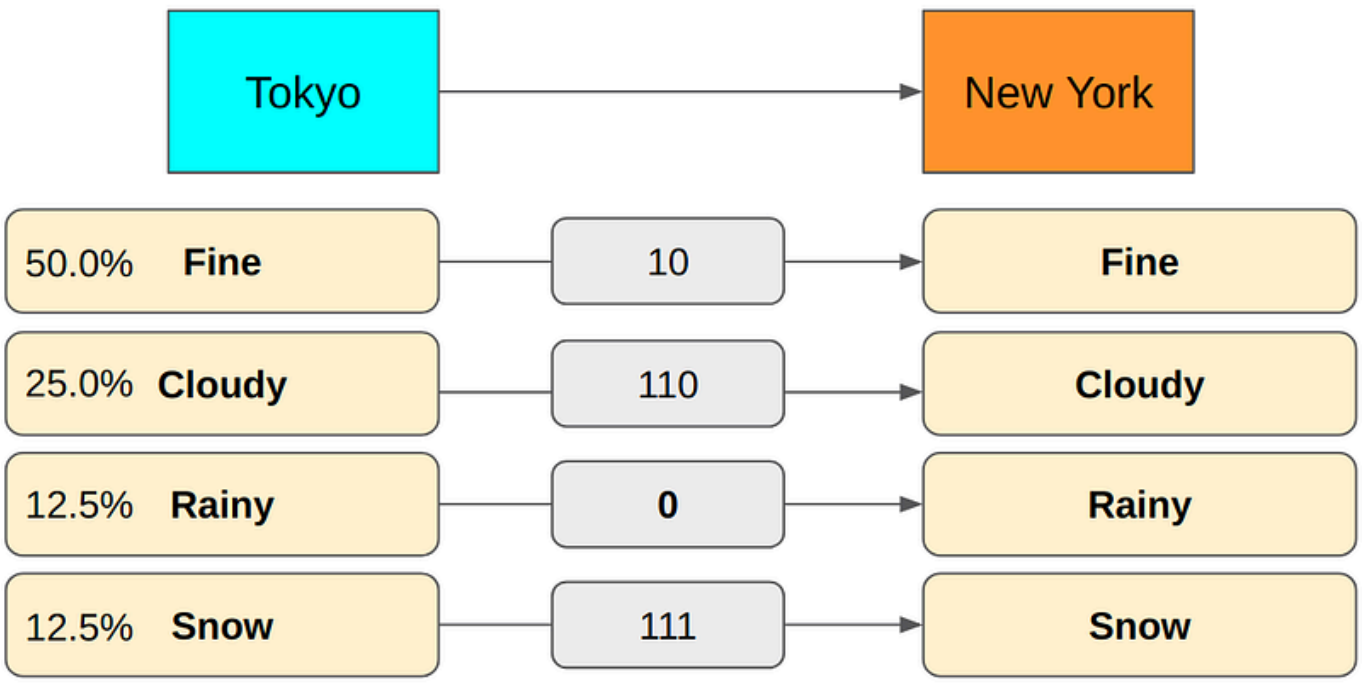
Let’s suppose we have the following probability distribution. How do we calculate the minimum lossless encoding size for each message type?
假设我们有以下概率分布。如何计算每种信息类型的最小无损编码大小?



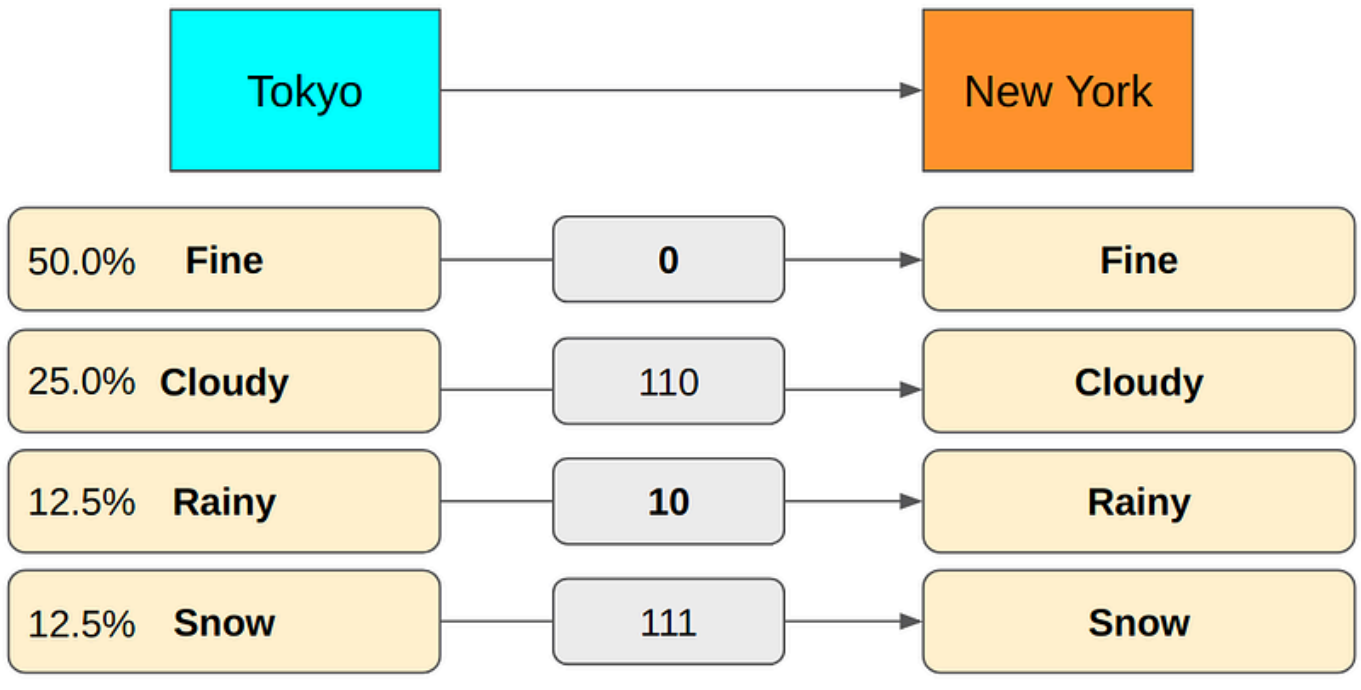

What is the minimum required encoding size for “Rainy”? How many bits should we use?
Rainy" 所需的最小编码大小是多少？我们应该使用多少比特?

Let’s suppose we use 1 bit to encode “Rainy” so that 0 means “Rainy”. In this case, we would have to use at least 2 bits for other message types to make the whole encoding unambiguous. It is not optimal as message types like “Fine” and “Cloudy” happens more frequently. We should reserve 1-bit encoding for them to make the average encoding size smaller. Hence, we should use more than 1 bit for “Rainy” as it occurs much less frequently.
假设我们用 1 位来编码 "Rainy"，那么 0 表示 "Rainy"。在这种情况下，我们就必须为其他信息类型使用至少 2 个比特，以使整个编码明确无误。由于 "晴朗 "和 "多云 "等信息类型出现的频率较高，这种编码方式并不理想。我们应该为它们保留 1 位编码，使平均编码大小更小。因此，我们应该为 "多雨 "使用多于 1 位的编码，因为它出现的频率要低得多。

As an example shown below, the encoding is unambiguous but the average size is not a theoretical minimum because we are using more bits for more frequent message types like “Fine” and “Cloudy”.
如下图所示，编码是明确的，但平均大小并不是理论上的最小值，因为我们对 "晴 "和 "多云 "等频率较高的信息类型使用了更多的比特。

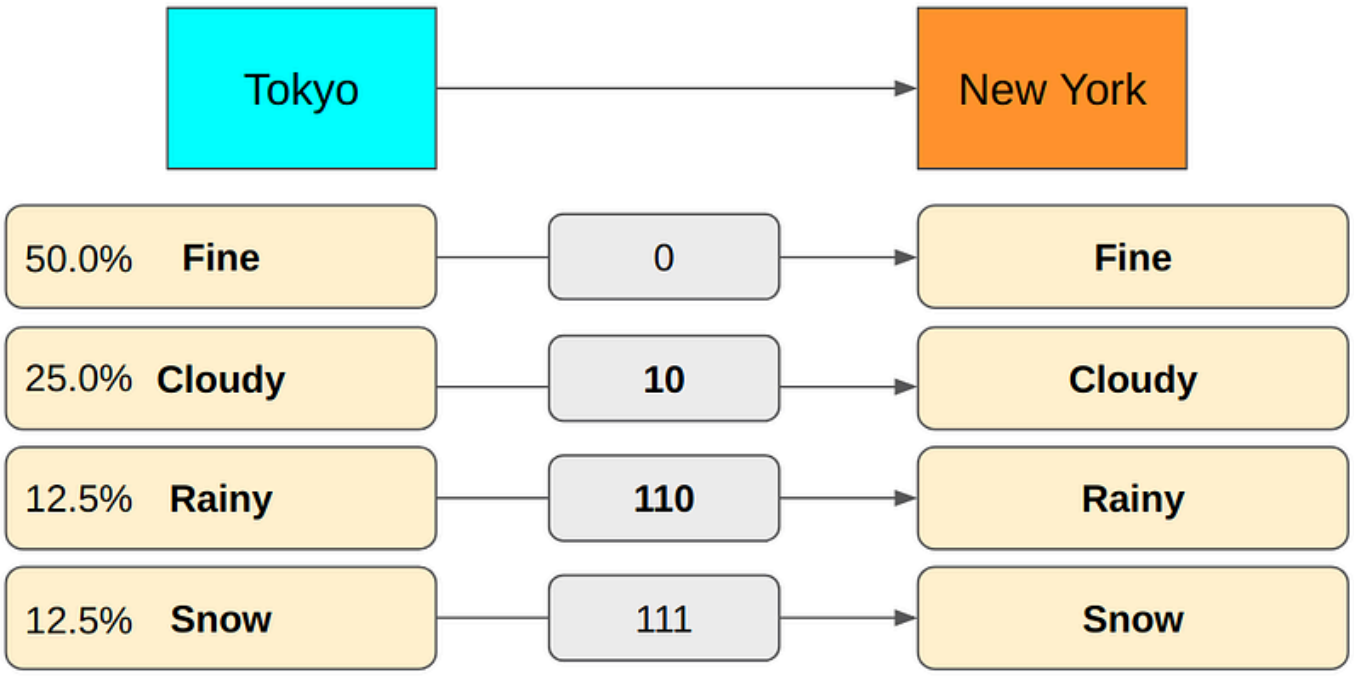



How about using 2 bits for “Rainy”? Let’s swap the encoding between “Rainy” and “Fine”.
用 2 比特来表示 "Rainy "如何？让我们交换一下 "Rainy "和 "Fine "的编码。



This encoding has the same problem as the previous one because we are using more bits for more frequent messages type (“Cloudy”).
这种编码方式与前一种编码方式存在同样的问题，因为我们使用更多的比特来处理更频繁的信息类型（"多云"）。

In fact, if you use 3 bits for “Rainy”, we can achieve the minimum encoding size on average.
事实上，如果用 3 比特来表示 "Rainy"，我们平均就能达到最小编码大小。

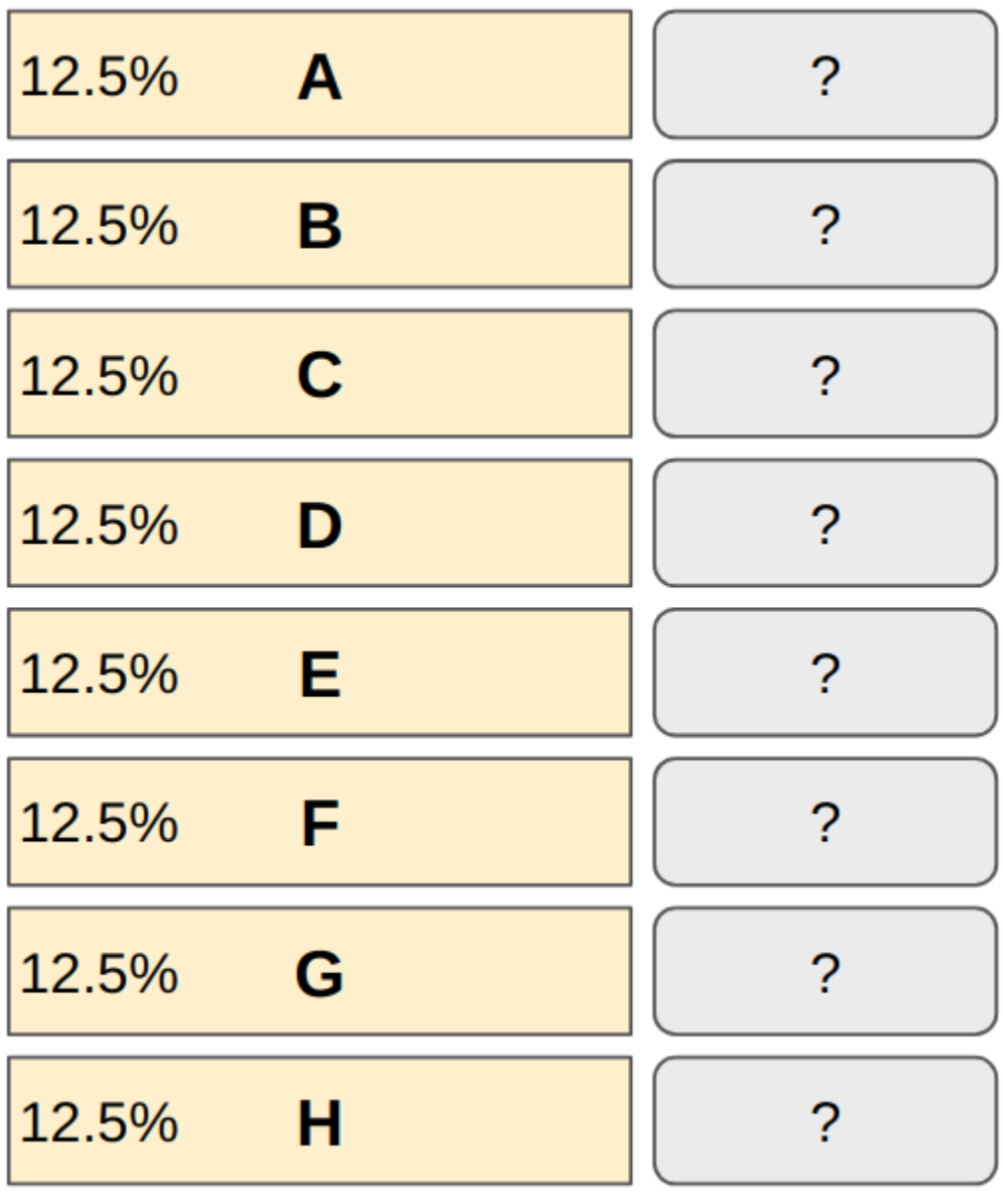



If we use 4 bits for “Rainy”, our encoding becomes redundant. So, 3 bits is the right encoding size for “Rainy”. It’s all good, but this process involves too many trials and errors to find the right encoding size.
如果 "Rainy "使用 4 比特，我们的编码就会变得多余。因此，3 比特才是 "Rainy "的正确编码大小。这一切都很好，但要找到正确的编码大小，这个过程涉及太多的试验和错误。

Given a probability distribution, is there any easy way to calculate the smallest possible average size of lossless encoding of the messages sent from the source to the destination. In other words, how do we calculate entropy?
在给定概率分布的情况下，有没有简单的方法可以计算出从源头发送到目的地的信息的无损编码的最小平均大小。换句话说，我们如何计算熵?

## How to Calculate Entropy
## 如何计算熵

Suppose we have 8 message types, each of which happens with equal probability (⅛ = 12.5%). What is the minimum number of bits we need to encode them without any ambiguity?
假设我们有 8 种信息类型，每种类型发生的概率相同（⅛ = 12.5%）。要对它们进行编码而不产生歧义，至少需要多少比特?



How many bits do we need to encode 8 different values?
编码 8 个不同的值需要多少比特?

1 bit can encode 0 or 1 (2 values). Adding another bit doubles the capacity since 2 bits = 2x2 = 4 values (00, 01, 10, and 11). So, 8 different values require 3 bits = 2x2x2 = 8 (000, 001, 010, 011, 100, 101, 110 and 111).
1 个比特可以编码 0 或 1（2 个值）。由于 2 位 = 2x2 = 4 个值（00、01、10 和 11），再增加一位，容量就会翻倍。因此，8 个不同的值需要 3 位 = 2x2x2 = 8（000、001、010、011、100、101、110 和 111）。

| | | |
|---|---|---|
| 12.5% | **A** | 000 |
| 12.5% | **B** | 001 |
| 12.5% | **C** | 010 |
| 12.5% | **D** | 011 |
| 12.5% | **E** | 100 |
| 12.5% | **F** | 101 |
| 12.5% | **G** | 110 |
| 12.5% | **H** | 111 |

We can not reduce any bit from any of the message types. If we do, it will cause ambiguity. Also, we don’t need more than 3 bits to encode 8 different values.
我们不能减少任何信息类型中的任何位。否则会引起歧义。此外，我们不需要超过 3 个比特来编码 8 个不同的值。

In general, when we need N different values expressed in bits, we need
一般来说，当我们需要用比特表示 N 个不同的值时，我们需要

$$\log_2 N$$

bits and we don’t need more than this.
我们不需要更多了。

For example,
例如

$$\log_2 8 = \log_2 2^3 = 3 \text{ bits}$$

In other words, if a message type happens 1 out of N times, the above formula gives the minimum size required. As P=1/N is the probability of the message, the same thing can be expressed as:
换句话说，如果一种信息类型在 N 次中出现 1 次，上式就给出了所需的最小大小。由于 P=1/N 是信息发生的概率，因此可以表示为：

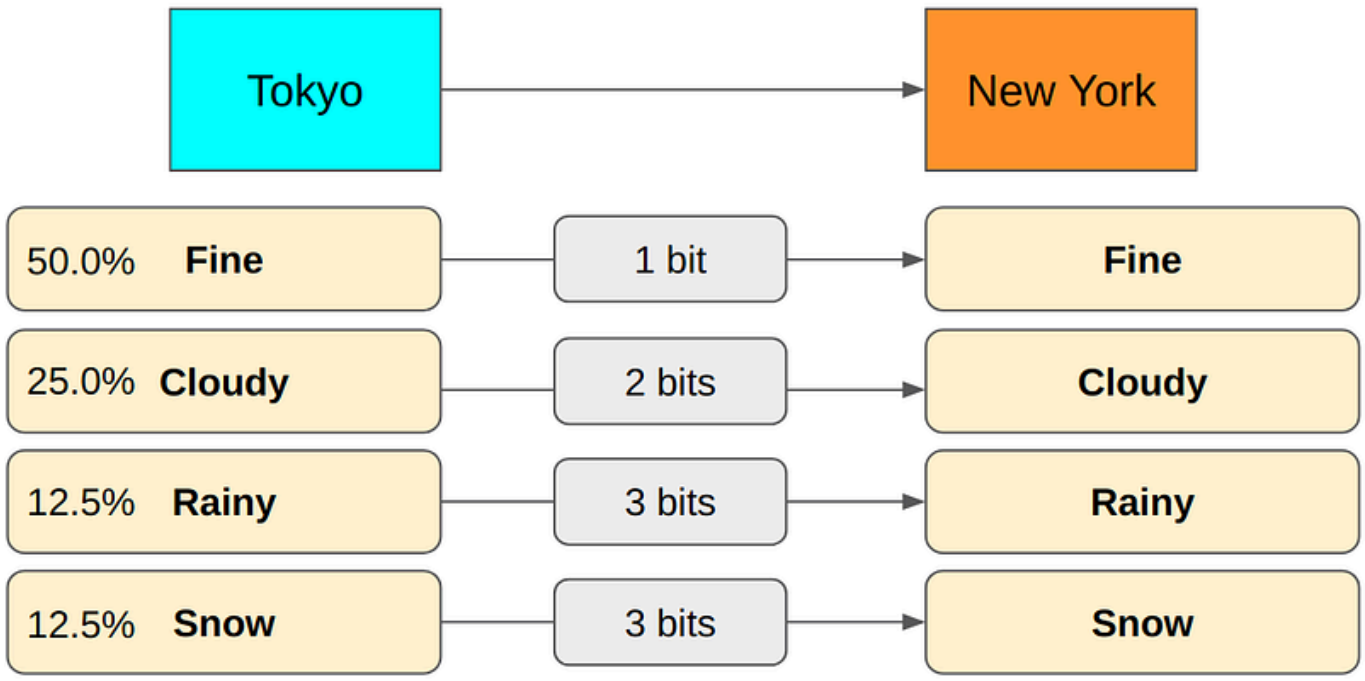
$$\log_2 N = -\log_2 1/N = -\log_2 P$$

Let’s combine this knowledge with how we calculate the minimum average encoding size (in bits) which is entropy:
让我们把这些知识与计算最小平均编码大小（以比特为单位）的方法结合起来，这就是熵：

$$Entropy = -\sum_i P(i)\, log_2 P(i)$$

P(i) is the probability of i-th message type. You may need to sit back and reflect on the simplicity and the beauty of this formula. There is no magic. We are merely calculating the average encoding size using the minimum encoding size for each message type.
P(i) 是第 i 种信息类型的概率。您可能需要静下心来思考一下这个公式的简洁性和优美性。这并不神奇。我们只是用每种报文类型的最小编码大小来计算平均编码大小。

Let's go through an example using the weather reporting scenario to solidify our understanding.
让我们以天气预报为例，加深理解。




P(Fine) = 0.5

$-\log_2 P(\text{Fine}) = -\log_2 0.5 = -\log_2 1/2 = \log_2 2 = 1 \text{ bit}$

P(Cloudy) = 0.25

$-\log_2 P(\text{Cloudy}) = -\log_2 0.25 = -\log_2 1/4 = \log_2 4 = 2 \text{ bits}$

P(Rainy) = 0.125

$-\log_2 P(\text{Rainy}) = -\log_2 0.125 = -\log_2 1/8 = \log_2 8 = 3 \text{ bits}$

P(Snow) = 0.125

$-\log_2 P(\text{Snow}) = -\log_2 0.125 = -\log_2 1/8 = \log_2 8 = 3 \text{ bits}$

So, the entropy is:
因此，熵为

(0.5 x 1 bit)+(0.25 x 2 bits)+(0.125 x 3 bits)+(0.125 x 3 bits)=1.75 bits
(0.5 x 1 位)+(0.25 x 2 位)+(0.125 x 3 位)+(0.125 x 3 位)=1.75 位

## What About Those Analogies?
## 那些比喻是怎么回事?

There are a lot of analogies used for entropy: disorder, uncertainty, surprise, unpredictability, amount of information and so on.
关于熵，有很多类比：无序、不确定性、惊喜、不可预测性、信息量等等。

If entropy is high (encoding size is big on average), it means we have many message types with small probabilities. Hence, every time a new message arrives, you'd expect a different type than previous messages. You may see it as a disorder or uncertainty or unpredictability.
如果熵很高（平均编码大小很大），这意味着我们有很多概率很小的报文类型。因此，每次有新信息到来时，你都会期待与之前不同的信息类型。你可能会认为这是一种无序、不确定性或不可预测性。

When a message types with much smaller probability than other message types happens, it appears as a surprise because on average you'd expect other more frequently sent message types.
当一种信息类型发生的概率比其他信息类型小得多时，会让人感到意外，因为平均而言，你会想到其他更频繁发送的信息类型。

Also, a rare message type has more information than more frequent message types because it eliminates a lot of other probabilities and tells us more specific information.
此外，罕见的信息类型比更频繁的信息类型有更多的信息，因为它消除了很多其他概率，告诉我们更具体的信息。

In the weather scenario, by sending “Rainy” which happens 12.5% of the times, we are reducing the uncertainty by 87.5% of the probability distribution (“Fine, Cloudy, Snow”) provided we had no information before. If we were sending “Fine” (50%) instead, we would be reducing the uncertainty by 50% only.
在天气情景中，如果我们发送 "下雨"（12.5% 的情况下会发生），那么我们就将概率分布（"晴、多云、下雪"）的不确定性降低了 87.5%，前提是我们之前没有任何信息。如果我们发送的是 "晴天"（50%），那么我们将只减少 50%的不确定性。

If the entropy is high, the average encoding size is significant which means each message tends to have more (specific) information. Again, this is why high entropy is associated with disorder, uncertainty, surprise, unpredictability, amount of information.
如果熵值高，平均编码大小就大，这意味着每条信息往往包含更多（具体的）信息。这也是为什么高熵与无序、不确定性、惊喜、不可预测性和信息量有关。

Low entropy means that most of the times we are receiving the more predictable information which means less disorder, less uncertainty, less surprise, more predictability and less (specific) information.
低熵意味着大多数时候我们接收到的是更可预测的信息，这意味着更少的无序、更少的不确定性、更少的惊喜、更多的可预测性和更少（具体）的信息。

I hope these analogies are no longer confusing for you.
希望这些类比不再让你感到困惑。